
Answers to Selected Problems

CHAPTER 1

1.2 (a) 16,384 (b) 33,554,432 (c) 2,147,483,648

1.3 (a) $(1203)_4 = 99_{10}$ (b) $(5243)_6 = 1179_{10}$

1.5 (a) 8 (b) 7 (c) 4

1.6 6

1.7 $CA5E_{16} = 1100_1010_0101_1110_2 = 1_100_101_001_011_110_2 = (145136)_8$

1.9 (a) $(21.625)_{10}$ (b) $(100.5)_{10}$ (c) $(177.5625)_{10}$

1.12 (a) Sum: 10011 Product: 1001110 (b) Sum: EF Product: 1930

1.19 (a) 010087 (b) 008485 (c) 991515 (d) 989913

1.24	(a)	5	2	1	1	Decimal
		0	0	0	0	0
		0	0	0	1	1
		0	0	1	1	2
		0	1	0	1	3
		0	1	1	1	4
		1	0	0	0	5
		1	0	0	1	6
		1	0	1	1	7
		1	1	0	1	8
		1	1	1	1	9

- 1.29 C. BABBAGE
 1.31 $62 + 32 = 94$ printing characters; 34 special characters
 1.32 Complement bit 6 from the right
 1.33 (a) 8,759 (b) 5,426 (c) F56E (d) BEL; Y

CHAPTER 2

- 2.2 (a) x (b) x (c) y (d) 0
 2.3 (a) y (b) $z(x + y)$ (c) $x'y'$ (d) $x(w + y)$ (e) 0
 2.4 (a) $y + z'$ (b) $y(x' + z)$ (c) x' (d) $xy(w + z)$
 2.9 (a) $(x' + y' + z')(x + y + z)$
 2.11 (a) $F(x, y, z) = \Sigma(0, 2, 3, 6, 7)$
 2.12 (a) 10000010 (c) 01011001 (d) 00110101
 2.14 (b) $(x + y')' + (x' + y)' + (x' + z')'$
 2.15 $T_1 = C + A'B$
 $T_2 = A'C' + AC$
 2.17 (a) $\Sigma(5, 7, 11, 12, 13, 14, 15) = \Pi(0, 1, 2, 3, 4, 6, 8, 9, 10)$
 2.18 (c) $F = xy + w'z + y'z$
 2.19 $F = \Sigma(6, 7, 8, 9, 10, 11, 14, 15) = \Pi(0, 1, 2, 3, 4, 5, 12, 13)$
 2.22 (a) $wx + wy'z + xy' = (w + x)(w + y')(x + y')(x + z)$
 (b) $x'y' + w'z' + y'z' = (x' + z')(y' + z')(w' + y')$

CHAPTER 3

- 3.1 (a) $x'z' + yz$ (b) $y + xz$ (c) $z' + x'y'$ (d) z
 3.2 (a) $x'y + xy' + xz$ (b) $x + y'$
 3.3 (a) $x'z + xy$ (b) $x + y'z'$ (c) $xy' + x'y + yz'$
 3.4 (a) $y'z' + w'y'$ (b) $xz' + wx'z$ (c) $w'z + x'z + w'xy$
 (d) $yz' + xy$
 3.5 (a) $y'z' + wx'y' + w'x'y + w'yz'$ (b) $x'y'z' + w'x' + xy'z$
 (d) $w'x'z + wy' + w'y$
 3.6 (a) $AC'D' + B'C' + AB'$ (b) $y'z' + x'y' + w'x'z$
 3.7 (a) $x + w'z$ (c) $BC' + AC' + ABD$
 3.8 (a) $F(w, x, y, z) = \Sigma(4, 5, 6, 7, 11, 13, 15)$
 (b) $F(A, B, C, D) = \Sigma(3, 5, 7, 11, 12, 13, 14, 15)$

3.9 (a) Essential: xz and $x'z'$; Nonessential: $w'x$ and $w'z'$

(b) $F = B'D' + AC + A'BD + (CD \text{ or } B'C)$

3.10 (c) $F = BC' + AC + A'B'D$

Essential: BC' , AC

Nonessential: AB , $A'B'D$, $B'CD$, $A'C'D$

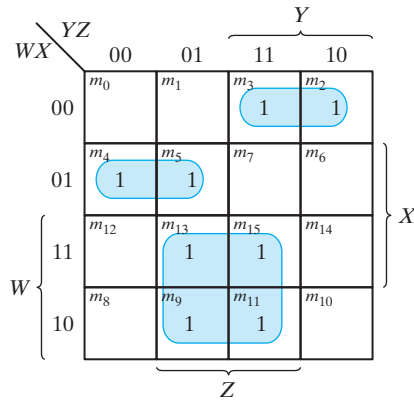
3.11 $F = (y' + z')(w + x' + y')(x + y + z)(w' + y + z)$

3.12 (a)

$F(A, B, C, D) = \Pi(0, 1, 6, 7, 8, 10, 12, 14)$

$F'(A, B, C, D) = \Sigma(0, 1, 6, 7, 8, 10, 12, 14)$

$F(A, B, C, D) = \Sigma(2, 3, 4, 5, 9, 11, 13, 15)$



$F' = w'x'y + w'xy' + wz$

3.13 (a) $F = x'y' + xz = (x + z')(x' + y)$

3.15 (b) $F(A, B, C, D) = A' + BC'D' = \Sigma(1, 2, 4, 6, 12)$

3.17 $F' = A'B'D' + BC + AB$

3.19 (a) $F = (y + (w + x)')$

3.30 $F = (A \oplus B)(C \oplus D)$

3.35

Line 1: Dash not allowed, use underscore: Exmpl_3.

Terminate line with semicolon (;).

Line 2: **inputs** should be **input** (no s at the end).

Change last comma (,) to semicolon (;). Output is declared but does not appear in the port list, and should be followed by a comma if it is intended to be in the list of inputs. If *Output* is a misspelling of **output** and is to declare output ports, C should be followed by a semicolon (;) and *F* should be followed by a semicolon (;).

Line 3: *B* cannot be declared as input (Line 2) and output (Line 3). Terminate the line with a semicolon (;).

Line 4: *A* cannot be an output of the primitive if it is an input to the module.

Line 5: Too many entries for the not gate (only two allowed).

Line 6: OR must be in lowercase: change to “or”.

Line 7: **endmodule** is misspelled. Remove semicolon (no semicolon after endmodule).

CHAPTER 4

4.1 (a) $F_1 = A + B'C + BD' + B'D$

$$F_2 = A'B + D$$

4.2 $F = ABC + A'D$

$$G = ABC + A'D'$$

4.3 (b) 1024 rows and 4 columns

4.4 (a) $F = x + yz$

4.6 (a) $F = x'y' + x'z' + y'z'$

4.7 (a) $w = A \quad x = A \oplus B \quad y = x \oplus C \quad z = y \oplus D$

4.8 (a) The 8-4-2-1 code (Table 1.5) and the BCD code (Table 1.4) are identical for digits 0 – 9.

4.10 Inputs: A, B, C, D ; Outputs: w, x, y, z

$$z = D$$

$$y = C \oplus D$$

$$x = B \oplus (C + D)$$

$$w = A \oplus (B + C + D)$$

4.12 (b) $Diff = x \oplus y \oplus B_{in}$

$$B_{out} = x'y + x'B_{in} + yB_{in}$$

4.13	Sum	C	V
(a)	1101	0	1
(b)	0001	1	1
(c)	0100	1	0
(d)	1011	0	1
(e)	1111	0	0

4.14 30 ns

4.18 $w = A'B'C'$

$$x = B \oplus C$$

$$y = C$$

$$z = D'$$

4.22 $w = AB + ACD$

$$x = B'C' + B'D' + BCD$$

$$y = C'D + CD'$$

$$z = D'$$

4.28 (a) $F_1 = \Sigma(0, 1, 5, 7)$

$$F_2 = \Sigma(3, 4, 7)$$

$$F_3 = \Sigma(0, 6, 7)$$

4.29 $x = D_0'D_1'$

$$y = D_0'D_1 + D_0'D_2'$$

4.34 (a) $F(A, B, C, D) = \Sigma(1, 4, 5, 6, 9, 10, 11, 12, 13)$

4.35 (a) When $AB = 00, F = D$

$$\text{When } AB = 01, F = (C + D)'$$

$$\text{When } AB = 10, F = CD$$

$$\text{When } AB = 11, F = 1$$

4.39 (a) // Verilog 1995

```

module Compare (A, B, Y);
  input  [3: 0]  A, B;    // 4-bit data inputs.
  output [5: 0]  Y;      // 6-bit comparator output.
  reg    [5: 0]  Y;      // EQ, NE, GT, LT, GE, LE

  always @ (A or B)
    if (A==B) Y = 6'b10_0011;    // EQ, GE, LE
    else if (A < B) Y = 6'b01_0101; // NE, LT, LE
    else      Y = 6'b01_1010;    // NE, GT, GE
endmodule

```

// Verilog 2001, 2005

```

module Compare (input [3: 0] A, B, output reg [5:0] Y);
  always @ (A, B)
    if (A==B)      Y = 6'b10_0011;    // EQ, GE, LE
    else if (A < B) Y = 6'b01_0101;    // NE, LT, LE
    else           Y = 6'b01_1010;    // NE, GT, GE
endmodule

```

// VHDL

```

entity Compare is
  port (A, B: in Std_Logic_vector 3 downto 0; Y: out Std_Logic_Vector 5 downto 0);
end Compare;

architecture Behavioral of Compare is
begin
  process (A, B) begin
    if A = B      then Y <= '10_0011';    // EQ, GE, LE
    elsif A < B   then Y <= '01_0101';    // NE, LT, LE
    else         Y <= '01_1010; end if;    // NE, GT, GE
  end Behavioral;

```

4.42 (c)

```

module Xs3_Behavior_95 (A, B, C, D, w, x, y, z);
  input A, B, C, D;
  output w, x, y, z;
  reg w, x, y, z;

  always @ (A or B or C or D) begin {w, x, y, z} = {A, B, C, D} + 4'b0011; end
endmodule

module Xs3_Behavior_2001 (input A, B, C, D, output reg w, x, y, z);
  always @ (A, B, C, D) begin {w, x, y, z} = {A, B, C, D} + 4'b0011; end
endmodule

entity Xs3_Behavior_vhdl is
  port (A, B, C, D: in std_logic; w, x, y, z: out std_logic);
end Xs3_Behavior_vhdl;

architecture Behavioral of Xs3_Behavior_vhdl is
begin
  w & x & y & z <= A & B & C & D + '0011';
end Behavioral;

```

4.50 (a) 8-4-2-1 to BCD code converter

// See Problem 4.8 and Table 1.5.

```

module Prob_4_50a (output reg [3: 0] Code_BCD, input [3: 0] Code_84_m2_m1);
  always @ (Code_84_m2_m1)
    case (Code_84_m2_m1)
      4'b0000: Code_BCD = 4'b0000;    // 0
      4'b0111: Code_BCD = 4'b0001;    // 1
      4'b0110: Code_BCD = 4'b0010;    // 2
      4'b0101: Code_BCD = 4'b0011;    // 3
      4'b0100: Code_BCD = 4'b0100;    // 4
      4'b1011: Code_BCD = 4'b0101;    // 5
      4'b1010: Code_BCD = 4'b0110;    // 6
      4'b1001: Code_BCD = 4'b0111;    // 7
      4'b1000: Code_BCD = 4'b1000;    // 8
      4'b1111: Code_BCD = 4'b1001;    // 9

      4'b0001: Code_BCD = 4'b1010;    // 10
      4'b0010: Code_BCD = 4'b1011;    // 11
      4'b0011: Code_BCD = 4'b1100;    // 12
      4'b1100: Code_BCD = 4'b1101;    // 13
      4'b1101: Code_BCD = 4'b1110;    // 14
      4'b1110: Code_BCD = 4'b1111;    // 15
    endcase
endmodule

entity Prob_4_50a_vhdl is
port (code_BCD: out std_logic_vector (3 downto 0); Code_84_md_m1: in std_logic
      _vector (3 downto 0));
end Prob_4_50a_vhdl;

architecture Behavioral of Prob_4_50a_vhdl is
begin
  process (Code_84_m2_m1)
    case (Code_84_m2_m1) is
      when '0000' => Code_BCD <= '0001';
      when '0110' => Code_BCD <= '0010';
      when '0101' => Code_BCD <= '0011';
      when '0100' => Code_BCD <= '0100';
      when '1011' => Code_BCD <= '0101';
      when '1010' => Code_BCD <= '0110';
      when '1001' => Code_BCD <= '0111';
      when '1000' => Code_BCD <= '1000';
      when '1111' => Code_BCD <= '1001';
    endcase
  endprocess

```

```

        when '0001' => Code_BCD <= '1010';
        when '0010' => Code_BCD <= '1011';
        when '0011' => Code_BCD <= '1100';
        when '1100' => Code_BCD <= '1101';
        when '1101' => Code_BCD <= '1110';
        when '1110' => Code_BCD <= '1111';
    endcase;
end process;
end Behavioral;

```

4.56 Verilog: **assign** match = (A == B); // Assumes **reg** [3: 0] A, B;
 VHDL: match <= (A = B);

4.57 **module** Prob_4_57(
 input D0, D1, D2, D3,
 output reg x_in, y_in, Valid
);

```

always @ (D0, D1, D2, D3) begin
    case ({D0, D1, D2, D3})
        4'b0000: {x_out, y_out, Valid} = 3'bxx0;
        4'b1xxx,: {x_out, y_out, Valid} = 3'b001;
        4'b01xx: {x_out, y_out, Valid} = 3'b011;
        4'b001x: {x_out, y_out, Valid} = 3'b101;
        4'b0001: {x_out, y_out, Valid} = 3'b111;
    endcase
end
endmodule

```

entity Prob_4_57 **is**
 port (D0, D1, D2, D3: **in** Std_Logic; x_out, y_out: **out** Std_Logic; Valid: **out** Std_Logic);
end Prob_4_57;

```

architecture Behavioral of Prob_4_57 is
begin
    x_out & y_out & Valid <= "000" when D0 & D1 & D2 & D3 = '0000'; else
    x_out & y_out & Valid <= "001" when D0 = 1; else
    x_out & y_out & Valid <= "011" when D0 = 0 and D1 = 1; else
    x_out & y_out & Valid <= "101" when D0 = 0 and D1 = 0 and D2 = 1; else
    x_out & y_out & Valid <= "111" when D0 & D1 & D2 & D3 = '0001'; end if;
endcase;
end process;
end Behavioral;

```


CHAPTER 5

5.4 (c) $A'Q' + BQ$

5.7 $S = x \oplus y \oplus Q$

$$Q(t+1) = xy + xQ + yQ$$

5.8 A counter with a repeated sequence of 00, 01, 10

5.9 (a) $A(t+1) = x'A' + BA$

$$B(t+1) = AB' + x'B$$

5.11 (a)

Present state:	00	01	11	00	00	01	00	01	00	00	01	11	10	00	01
Input:	1	1	0	0	1	0	1	0	0	1	1	1	0	1	0
Output:	0	0	1	0	0	1	0	1	0	0	0	0	1	0	1
Next state:	01	11	00	00	01	00	01	00	00	01	11	10	00	01	00

5.12 (b)

Present state	Next state		Output	
	0	1	0	1
<i>a</i>	<i>f</i>	<i>b</i>	0	0
<i>b</i>	<i>d</i>	<i>a</i>	0	0
<i>d</i>	<i>g</i>	<i>a</i>	1	0
<i>f</i>	<i>f</i>	<i>b</i>	1	1
<i>g</i>	<i>g</i>	<i>d</i>	0	1

5.13 (a)

State:	<i>a f b a b d g d g g d a</i>
Input:	0 1 1 1 0 0 1 0 0 1 1
Output:	0 1 0 0 0 1 1 1 0 1 0

(b)

State:	<i>a f b c e d g h g g h a</i>
Input:	0 1 1 1 0 0 1 0 0 1 1
Output:	0 1 0 0 0 1 1 1 0 1 0

5.16 (a) $D_A = Ax' + Bx$
 $D_B = A'x + Bx'$

5.18 $J_A = K_A = (BF + B'F')E$
 $J_B = K_B = E$

5.19 (a) $D_A = A'B'x_{in}$
 $D_B = A + C'x_{in}' + BCx_{in}$
 $D_C = Cx_{in}' + Ax_{in} + A'B'x_{in}'$
 $y_{out} = A'x_{in}$

5.23 (a) $RegA = 10$, $RegB = 10$

(b) $RegA = 10$, $RegB = 20$

5.26 (a)

$$Q(t + 1) = JQ' + K'Q$$

When $Q = 0$, $Q(t + 1) = J$

When $Q = 1$, $Q(t + 1) = K'$

```

module JK_Behavior (output reg Q, input J, K, CLK);
  always @ (posedge CLK)
    if (Q == 0)      Q <= J;
    else             Q <= ~K;
endmodule

```

5.31 (a)

```

module Seq_Ckt (input A, B, C, CLK, output reg Q);
  reg E;
  always @ (posedge CLK)
    begin
      Q = E & C;
      E = A | B;
    end
endmodule

```

(b)

```

process (CLK) begin
  if CLK'event and CLK = '1' then begin
    Q := E and C;
    E := A or B;
  end if;
end process;

```

CHAPTER 6

6.4 1010; 1101; 0110; 0011; 1001; 1100; 0110; 1011

6.8 $A = 0010, 0001, 1000, 1100$. Carry = 1, 1, 1, 0

6.9 (b) $J_Q = x'y$; $K_Q = (x' + y)'$

6.14 (a) 3

6.15 32 ns; 31.25 MHz

6.16 $1010 \rightarrow 1011 \rightarrow 0100$

$1100 \rightarrow 1101 \rightarrow 0100$

$1110 \rightarrow 1111 \rightarrow 0000$

6.17 $J_0 = K_0 = E$

$J_1 = K_1 = A'_0 E$

$J_2 = K_2 = A'_0 A'_1 E$

$J_3 = K_3 = A'_0 A'_1 A'_2 E$

6.19 (b) $D_{Q1} = Q'_1$

$D_{Q2} = Q_2 Q'_1 + Q'_8 Q'_2 Q_1$

$D_{Q4} = Q_4 Q'_1 + Q_4 Q'_2 + Q'_4 Q'_2 Q_1$

$D_{Q8} = Q_8 Q'_1 + Q_4 Q_2 Q_1$

6.21 $J_{A0} = LI_0 + L'C$

$K_{A0} = LI'_0 + L'C$

6.24 $T_A = A \oplus B$

$T_B = B \oplus C$

$T_C = AC + A'C' \quad (\text{not self-starting})$

$= AC + A'B'C \quad (\text{self-starting})$

6.26 The clock generator has a period of 8 ns. Use a 3-bit counter to count eight pulses.

6.28 (a) $D_A = A \oplus B$

$D_B = AB' + C$

$D_C = A'B'C'$

6.34 Verilog

module Shiftreg (SI, SO, CLK);

input SI, CLK;

output SO;

reg [3: 0] Q;

assign SO = Q[0];

always @ (posedge CLK)

 Q = {SI, Q[3: 1]};

endmodule

// Test plan

//

// Verify that data shift through the register

// Set SI = 1 for 4 clock cycles

// Hold SI = 1 for 4 clock cycles

// Set SI = 0 for 4 clock cycles

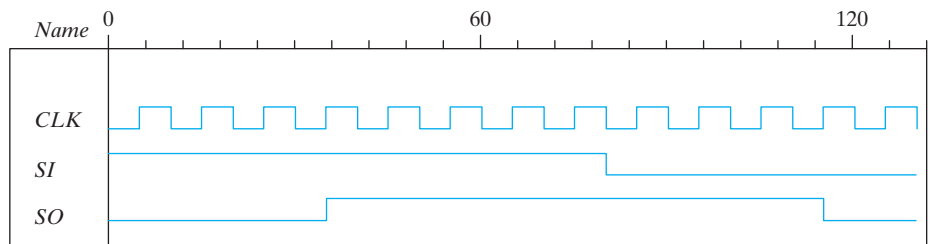
// Verify that data shifts out of the register correctly

```

module t_Shiftreg;
    reg SI, CLK;
    wire SO;

    Shiftreg M0 (SI, SO, CLK);

    initial #130 $finish;
    initial begin CLK = 0; forever #5 CLK = ~CLK; end
    initial fork
        SI = 1'b1;
        #80 SI = 0;
    join
endmodule
    
```



VHDL

```

entity Shiftreg is
    port (SI, CLK: in Std_Logic; SO: out Std_Logic);
end Shiftreg;

architecture Behavioral of Shiftreg is
    signal Q: Std_Logic_Vector (3 down to 0);
    begin
        SO <= Q(0);
        process (CLK) begin
            if CLK'event Q <= SI & Q(3: 1);
        end process;
    end Behavioral;

    entity t_Shiftreg is
    end t_Shiftreg;

    architecture Testbench of t_Shiftreg is
        component Shiftreg port (SI, CLK: in Std_Logic; SO: out Std_Logic);
        signal t_CLK, t_SI, t_SO: Std_Logic;
        begin
            UUT Shiftreg port map (SI => t_SI, CLK => t_CLK: SO => t_SO);
            t_SI <= '1';
            t_SI <= 0 after 80 ns
        
```

```

process begin
    t_CLK <= '0';
    wait for 5 ns;
    t_CLK <= '1';
    wait for 5 ns;
end process;
end Testbench;

```

6.35 (b) Verilog

```

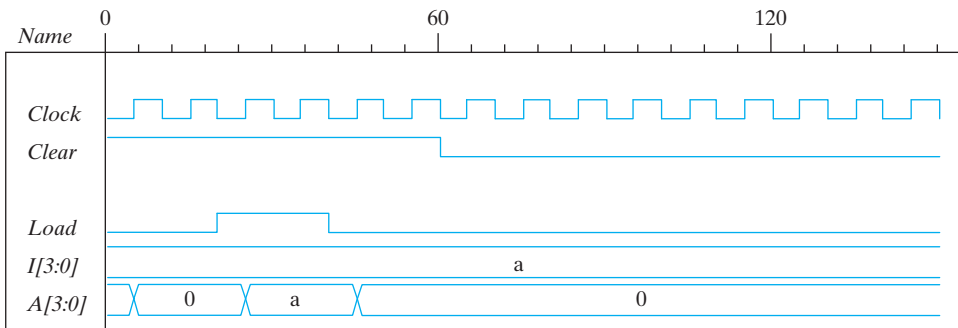
module Prob_6_35b (output reg [3: 0] A, input [3: 0] I, input Load, Clock, Clear);
    always @ (posedge Clock)
        if (Load) A <= I;
        else if (Clear) A <= 4'b0;
        //else A <= A;           // redundant statement
endmodule

module t_Prob_6_35b ( );

    wire [3: 0] A;
    reg [3: 0] I;
    reg Clock, Clear, Load;

    Prob_6_35b M0 (A, I, Load, Clock, Clear);
    initial #150 $finish;
    initial begin Clock = 0; forever #5 Clock = ~Clock; end
    initial fork
        I = 4'b1010; Clear = 1;
        #60 Clear = 0;
        Load = 0;
        #20 Load = 1;
        #40 Load = 0;
    join
endmodule

```



VHDL

```

entity Prob_6_35b is
  port (A: out std_logic_vector (3 downto 0); I: in std_logic_vector (3 downto 0),
        Load, Clock, Clear: in std_logic);
end Prob_6_35b;

architecture Behavioral of Prob_6_35b is
begin
process (Clock) begin
  if Clock'event and Clock = 1 then
    if Load = 1 then A <= I;
    elsif Clear = 1 then A <= "0000"; end if;
end process;
end Behavioral;

```

6.37 (a) Verilog

```

module Counter_if (output reg [3: 0] Count, input clock, reset);
  always @ (posedge clock , posedge reset)
    if (reset)Count <= 0;
    else if (Count == 0) Count <= 1;
    else if (Count == 1) Count <= 3; // Default interpretation is decimal
    else if (Count == 3) Count <= 7;
    else if (Count == 4) Count <= 0;
    else if (Count == 6) Count <= 4;
    else if (Count == 7) Count <= 6;
    else Count <= 0;
endmodule

```

VHDL

```

entity Counter is
  port (Count: out std_logic_vector (3 downto 0); clock, reset: in std_logic);
end Counter;

architecture Behavioral is
begin
process (clock, reset) begin
  if reset'event and reset = 1 then Count <= 0;
    elsif clock'event and clock = 1 then
      if Count = 0      then Count <= "1";
      elsif Count = 1  then Count <= "3";
      elsif Count == 3 then Count <= "7";
      elsif Count == 4 then Count <= "0";
      elsif Count == 6 then Count <= "4";
      elsif Count == 7 then Count <= "6";
      else Count <= 0;
    end if;
end process;
end Behavioral;

```

6.38 (a) Verilog

```

module Prob_6_38a_Updown (OUT, Up, Down, Load, IN, CLK);    // Verilog 1995
  output [3: 0]    OUT;
  input [3: 0]    IN;
  input            Up, Down, Load, CLK;
  reg [3:0]        OUT;

  always @ (posedge CLK)
  if (Load) OUT <= IN;
  else if (Up)     OUT <= OUT + 4'b0001;
  else if (Down)   OUT <= OUT - 4'b0001;
  else            OUT <= OUT;
endmodule

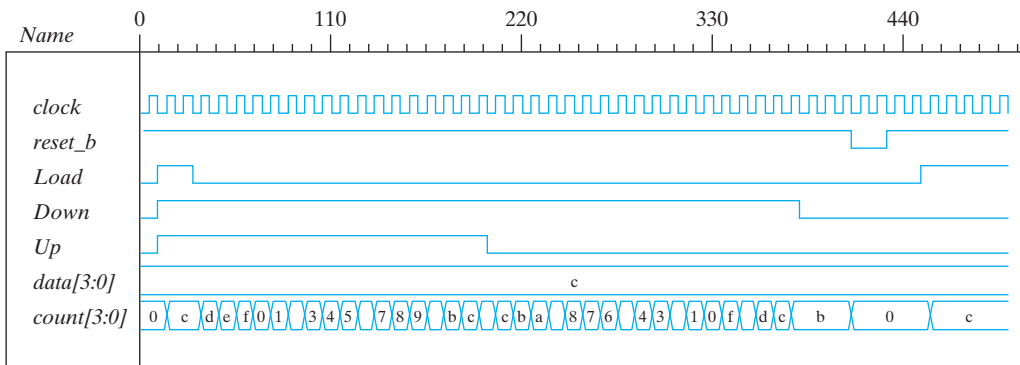
```

VHDL

```

entity Prob_6_38a is
  port (OUT_sig: out std_logic_vector (3 downto 0); IN_sig: in std_logic_vector
    (3 downto 0);
    Up, down, Load, CLK: in std_logic);
end Prob_6_38a;
architecture Behavioral of Prob_6_38a is
begin
  process (CLK) begin
    if CLK'event and CLK = 1 then
  if Load = 1 then OUT_sig <= IN_sig;
    elsif UP = 1 then OUT_sig <= OUT_sig + "0001";
    elsif DOWN = 1 then OUT_sig <= OUT_sig - "0001";
    else OUT_sig <= OUT_sig;
    end if;
  end process;
end Behavioral;

```



6.42 Verilog: Because A is a register variable, it retains whatever value has been assigned to it until a new value is assigned. Therefore, the statement $A_count \leq A_count$ has the same effect as if the statement was omitted.

VHDL: Because A_count is a signal and is assigned value by a procedural statement in a process, it retains its value until a subsequent assignment provides a different value. Therefore, the statement $A_count \leq A_count$ has the same effect as if the statement was omitted.

6.45

entity Prob_6_45 is

port (y_out: **out** std_Logic; start, clock, reset_bar: **in** Std_Logic);
end Prob_6_45;

architecture Behavioral is

constant

 s0 = "0000",
 s1 = "0001",
 s2 = "0010",
 s3 = "0011",
 s4 = "0100",
 s5 = "0101",
 s6 = "0110",
 s7 = "0111",
 s8 = "1000";

signal state, next_state: Std_Logic_Vector (3 **downto** 0);

begin

process (clock, reset_bar) **begin**

if reset_bar = 0 **then** state <= s0; **elsif** clock'event **and** clock = 1 **then**
 state <= next_state; **end if**;

end process;

process (state, start) **begin**

 y_out <= 1'b0;

case state **is**

when s0 => **if** (start) next_state <= s1; **else** next_state <= s0; **end if**;
when s1 => **begin** next_state <= s2; y_out <= 1; **end**
when s2 => **begin** next_state <= s3; y_out <= 1; **end**
when s3 => **begin** next_state <= s4; y_out <= 1; **end**
when s4 => **begin** next_state <= s5; y_out <= 1; **end**
when s5 => **begin** next_state <= s6; y_out <= 1; **end**
when s6 => **begin** next_state <= s7; y_out <= 1; **end**
when s7 => **begin** next_state <= s8; y_out <= 1; **end**
when s8 => **begin** next_state <= s0; y_out <= 1; **end**
others => next_state <= s0;

endcase;

end process;

end Behavioral;

- 6.50** **(b)** The HDL description is available on the Companion Website. Simulations results for Problem 6.50 follow:

CHAPTER 7

- 7.2** **(a)** 2^{18} **(b)** 2^{32} **(c)** 2^{25} **(d)** 2^{18}
- 7.3** Address: 01 0001 1011 = 011B (hex)
Data: 100 1011 1100 = 4BC (hex)
- 7.7** **(a)** 6×64 decoders, 128 AND gates **(b)** $x = 31; y = 33$
- 7.8** **(a)** 16 chips **(b)** 21; 17 **(c)** 4×16 decoder
- 7.10** 0 1111 0100 0111
- 7.11** 101 110 011 001 010
- 7.12** **(a)** 0101 1010; **(b)** 1011 1111; **(c)** 0101 0011
- 7.13** **(a)** 5 **(b)** 6 **(c)** 7
- 7.14** **(a)** 10110011001101
- 7.16** 34 pins
- 7.20** $M[5] = 1100; M[15] = 0100$
- 7.25** $A = yz' + xz' + x'y'z$
 $B = x'y' + yz + y'z'$
 $C = A + xyz$
 $D = z + x'y$

CHAPTER 8

- 8.1** **(a)** $R2 \leftarrow R1, R1 \leftarrow 0$

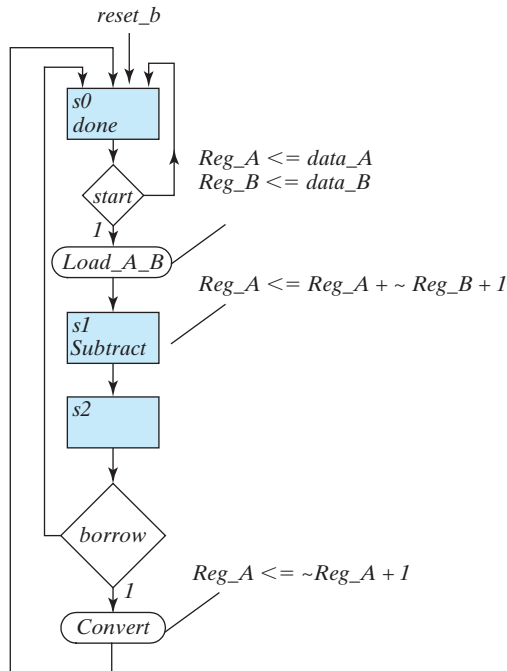
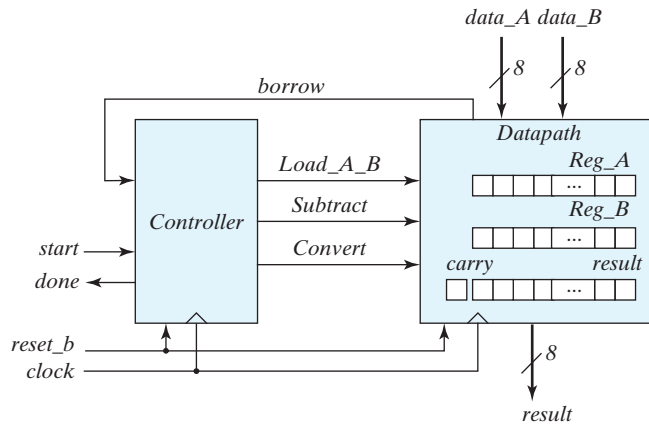
8.7 RTL notation:

S0: Initial state: if (start = 1) then ($RA \leftarrow data_A$, $RB \leftarrow data_B$, go to *S1*).

S1: {Carry, RA } $\leftarrow RA + (2\text{'s complement of } RB)$, go to *S2*.

S2: If (borrow = 0) goto *S0*. If (borrow = 1) then $RA \leftarrow (2\text{'s complement of } RA)$, go to *S0*.

Block diagram and ASMD chart:



Verilog

```
module Subtractor_P8_7
```

```
  (output done, output [7:0] result, input [7: 0] data_A, data_B, input start, clock,  
  reset_b);
```

```
  Controller_P8_7 M0 (Load_A_B, Subtract, Convert, done, start, borrow, clock, reset_b);
```

```
  Datapath_P8_7 M1 (result, borrow, data_A, data_B, Load_A_B, Subtract, Convert,  
  clock, reset_b);
```

```
endmodule
```

```
module Controller_P8_7 (output reg Load_A_B, Subtract, output reg Convert,  
output done,
```

```
  input start, borrow, clock, reset_b);
```

```
  parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10;
```

```
  reg [1: 0] state, next_state;
```

```
  assign done = (state == S0);
```

```
  always @ (posedge clock, negedge reset_b)
```

```
    if (!reset_b) state <= S0; else state <= next_state;
```

```
  always @ (state, start, borrow) begin Load_A_B = 0;
```

```
    Subtract = 0;
```

```
    Convert = 0;
```

```
  case (state)
```

```
    S0:   if (start) begin Load_A_B = 1; next_state = S1; end
```

```
    S1:   begin Subtract = 1; next_state = S2; end
```

```
    S2:   begin next_state = S0; if (borrow) Convert = 1; end
```

```
    default: next_state = S0;
```

```
  endcase
```

```
end
```

```
endmodule
```

```
module Datapath_P8_7 (output [7: 0] result, output borrow, input [7: 0] data_A,  
data_B,
```

```
  input Load_A_B, Subtract, Convert, clock, reset_b);
```

```
  reg carry;
```

```
  reg [8:0] diff;
```

```
  reg [7: 0] Reg_A, Reg_B;
```

```
  assign borrow = carry;
```

```
  assign result = RA;
```

```
always @ (posedge clock, negedge reset_b)
```

```
  if (!reset_b) begin carry <= 1'b0; Reg_A <= 8'b0000_0000; Reg_B <= 8'b0000_0000;
```

```
  end
```

```

else begin
  if (Load_A_B) begin Reg_A <= data_A; Reg_B <= data_B; end
  else if (Subtract) {carry, Reg_A} <= Reg_A + ~Reg_B + 1;

    // In the statement above, the math of the LHS is done to the word length of the LHS
    // The statement below is more explicit about how the math for subtraction is done:
    // else if (Subtract) {carry, Reg_A} <= {1'b0, Reg_A} + {1'b1, ~Reg_} + 9'b0000_0001;
    // If the 9th bit is not considered, the 2s complement operation will generate a carry bit,
    // and borrow must be formed as borrow = ~carry.

    else if (Convert) Reg_A <= ~Reg_A + 8'b0000_0001;
  end
endmodule

// Test plan – Verify;
// Power-up reset
// Subtraction with data_A > data_B
// Subtraction with data_A < data_B
// Subtraction with data_A = data_B
// Reset on-the-fly: left as an exercise

module t_Subtractor_P8_7;
  wire done;
  wire [7:0] result;
  reg [7:0] data_A, data_B;
  reg start, clock, reset_b;

  Subtractor_P8_7 M0 (done, result, data_A, data_B, start, clock, reset_b);

  initial #200 $finish;
  initial begin clock = 0; forever #5 clock = ~clock; end
  initial fork
    reset_b = 0;
    #2 reset_b = 1;
    #90 reset_b = 1;
    #92 reset_b = 1;
  join

  initial fork
    #20 start = 1;
    #30 start = 0;
    #70 start = 1;
    #110 start = 1;
  join

  initial fork
    data_A = 8'd50;
    data_B = 8'd20;

```

```

#50 data_A = 8'd20;
#50 data_B = 8'd50;

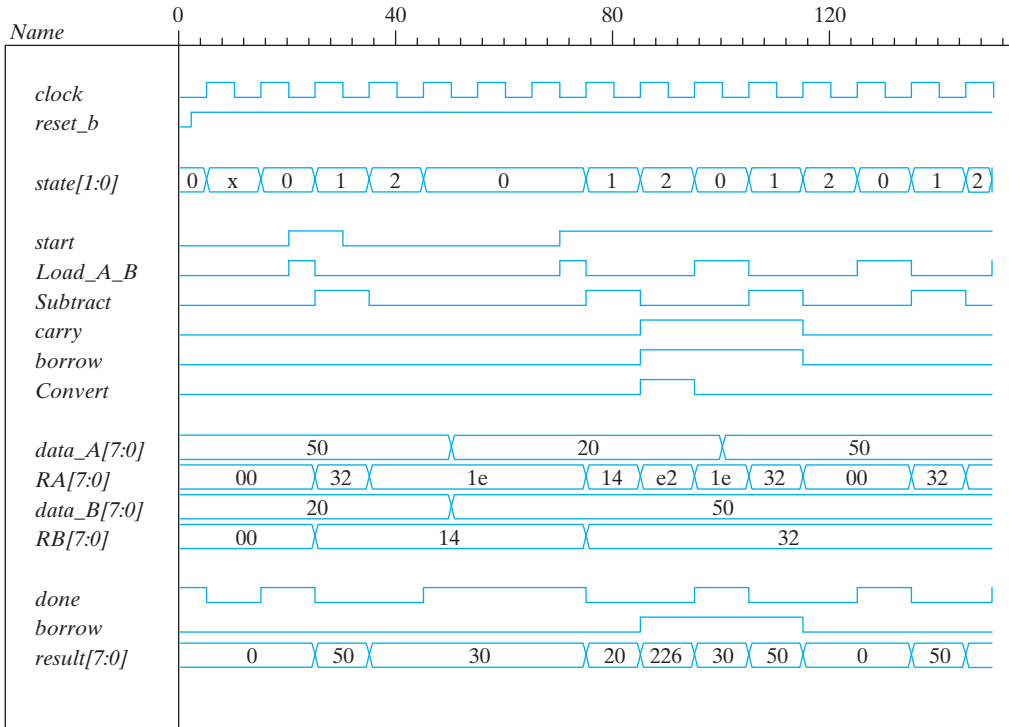
#100 data_A = 8'd50;
#100 data_B = 8'd50;

```

```

join
endmodule

```



VHDL

entity Datapath_P8_7 **is**

port (result: **out** std_logic_vector (7 **downto** 0);

data_a, data_b: **in** std_logic_vector (7 **downto** 0); Load_A_B, Subtract, Convert,
clock: **in** std_logic; borrow: **out** std_logic);

end Datapath_P8_7;

architecture Behavioral **of** Datapath_P8_7 **is**

begin

process (clock, reset_b) **begin**

if reset_b'event **and** reset_b = '0' **then** carry <= 0; Reg_A <= "00000000";
Reg_B <= "00000000";

elsif clock'event **and** clock = '1' **then**

if Load_A_B = 1 **then** Reg_A <= data_A; Reg_B <= data_B;

elsif Subtract = 1 **then** carry & Reg_A <= Reg_A + (not Reg_b) + 1;

elsif Convert **then** Reg_A <= (not Reg_A) + "00000001"; **end if**;

end Behavioral;

```

entity Controller_P8_7 is
  port (Load_A_B, Subtract, Convert, done: out Std_Logic; start, borrow, clock,
        reset_b: in Std_Logic);
end Controller_P8_7;

architecture Behavioral of Controller_P8_7 is
  constant S0 = "00", S1 = "01", S2 = "10";
  signal state, next_state: std_logic_vector (1 downto 0);
begin process (clock, reset_b)
begin
  if reset_b'event and reset_b = '0' then state <= S0;
  elsif clock'event and clock = '1' then state <= next_state; end if;
end process;
process (state, start, borrow)
begin

  Load_A_B <= 0;
  Subtract <= 0;
  Convert <= 0;
  case state is
    when S0 => if start = 1 then Load_A_B <= 1; next_state <= S1; end if;
    when S1 => Subtract <= '1'; next_state <= S2;
    when S2 => next_state <= S0; if borrow = 1 then convert = 1; end if;
    when others next_state <= S0;
  end process;
end Behavioral;

entity Subtractor_P8_7 is
  port (done: out std_logic; result: out std_logic_vector (7 downto 0);
        data_a, data_b: in std_logic_vector (7 downto 0); start, clock, reset_b: in std_logic);
end Subtractor_P8_7;

architecture ASMD is
component Controller_P8_7 is
  port (Load_A_B, Subtract, Convert, done: out Std_Logic; start, borrow, clock,
        reset_b: in Std_Logic);
end component;

component Datapath_P8_7 is
  port (result: out std_logic_vector (7 downto 0);
        data_a, data_b: in std_logic_vector (7 downto 0); Load_A_B, Subtract, Convert,
        clock: in
        std_logic; borrow: out std_logic);
end component;
begin
M0: Controller_P8_7
  port map (Load_A_B, Subtract, Convert, done, start, borrow, clock, reset_b);
M1: Datapath_P8_7
  port map (result, data_a, data_b, Load_A_B, Subtract, Convert, clock, borrow);
end ASMD;

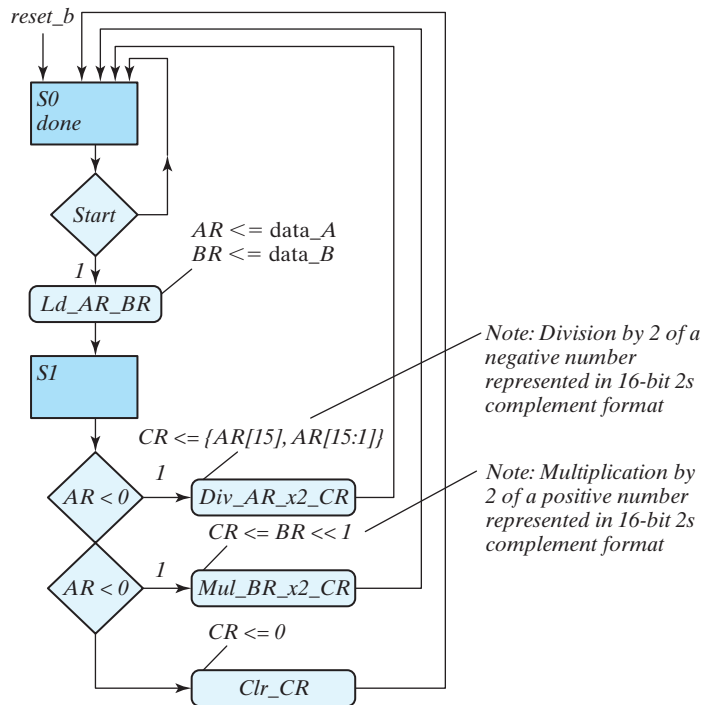
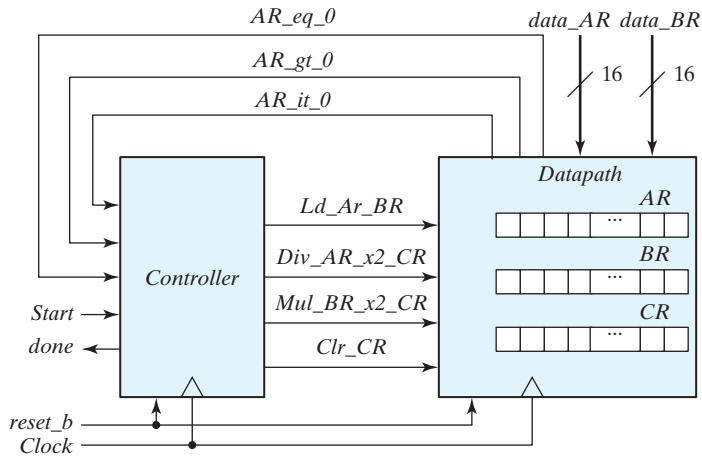
```

8.8 RTL notation:

S0: if (start = 1) $AR \leftarrow$ input data, $BR \leftarrow$ input data, go to *S1*.

S1: if ($AR[15] = 1$ (sign bit negative) then $CR \leftarrow AR$ (shifted right, sign extension).

else if (positive non-zero) then ($Overflow \leftarrow BR[15] \oplus [14]$), $CR \leftarrow BR$ (shifted left) else if ($AR = 0$) then ($CR \leftarrow 0$).



```
module Prob_8_8 (output done, input [15: 0] data_AR, data_BR, input start, clock,
reset_b);
```

```

Controller_P8_8 M0 (
    Ld_AR_BR, Div_AR_x2_CR, Mul_BR_x2_CR, Clr_CR, done,
    start, AR_lt_0, AR_gt_0, AR_eq_0, clock, reset_b
);
Datapath_P8_8 M1 (
    Overflow, AR_lt_0, AR_gt_0, AR_eq_0, data_AR, data_BR,
    Ld_AR_BR, Div_AR_x2_CR, Mul_BR_x2_CR, Clr_CR, clock, reset_b
);
endmodule

module Controller_P8_8 (
    output reg Ld_AR_BR, Div_AR_x2_CR, Mul_BR_x2_CR, Clr_CR,
    output done, input start, AR_lt_0, AR_gt_0, AR_eq_0, clock, reset_b
);

    parameter S0 = 1'b0, S1 = 1'b1;
    reg state, next_state;
    assign done = (state == S0);

    always @ (posedge clock, negedge reset_b)
        if (!reset_b) state <= S0; else state <= next_state;

    always @ (state, start, AR_lt_0, AR_gt_0, AR_eq_0) begin

        Ld_AR_BR = 0;
        Div_AR_x2_CR = 0;
        Mul_BR_x2_CR = 0;
        Clr_CR = 0;

        case (state)
            S0:      if (start) begin Ld_AR_BR = 1; next_state = S1; end
            S1:      begin
                        next_state = S0;
                        if (AR_lt_0) Div_AR_x2_CR = 1;
                        else if (AR_gt_0) Mul_BR_x2_CR = 1;
                        else if (AR_eq_0) Clr_CR = 1;
                    end

            default: next_state = S0;
        endcase
    end
endmodule

module Datapath_P8_8 (
    output reg Overflow, output AR_lt_0, AR_gt_0, AR_eq_0, input [15: 0] data_AR,
    data_BR,
    input Ld_AR_BR, Div_AR_x2_CR, Mul_BR_x2_CR, Clr_CR, clock, reset_b
);
    reg [15: 0] AR, BR, CR;
    assign AR_lt_0 = AR[15];
    assign AR_gt_0 = (!AR[15]) && (!AR[14:0]); // Reduction-OR
    assign AR_eq_0 = (AR == 16'b0);

```



```

always @ (posedge clock, negedge reset_b)
  if (!reset_b) begin AR <= 8'b0; BR <= 8'b0; CR <= 16'b0; end
  else begin
    if (Ld_AR_BR) begin AR <= data_AR; BR <= data_BR; end
    else if (Div_AR_x2_CR) CR <= {AR[15], AR[15:1]}; // For compiler without arithmetic
    right shift
    else if (Mul_BR_x2_CR) {Overflow, CR} <= (BR << 1);
    else if (Clr_CR) CR <= 16'b0;
  end
endmodule

// Test plan – Verify;
// Power-up reset
// If AR < 0 divide AR by 2 and transfer to CR
// If AR > 0 multiply AR by 2 and transfer to CR
// If AR = 0 clear CR
// Reset on-the-fly

module t_Prob_P8_8;
  wire      done;
  reg  [15: 0]  data_AR, data_BR;
  reg          start, clock, reset_b;
  reg  [15: 0]  AR_mag, BR_mag, CR_mag;    // To illustrate 2s complement math

// Probes for displaying magnitude of numbers
always @ (M0.M1.AR) // Hierarchical dereferencing via module path
  if (M0.M1.AR[15]) AR_mag = ~M0.M1.AR+ 16'd1; else AR_mag = M0.M1.AR;
always @ (M0.M1.BR )
  if (M0.M1.BR[15]) BR_mag = ~M0.M1.BR+ 16'd1; else BR_mag = M0.M1.BR;
always @ (M0.M1.CR)
  if (M0.M1.CR[15]) CR_mag = ~M0.M1.CR + 16'd1; else CR_mag = M0.M1.CR;

Prob_8_8 M0 (done, data_AR, data_BR, start, clock, reset_b);

initial #250 $finish;
initial begin clock = 0; forever #5 clock = ~clock; end
initial fork
  reset_b = 0;           // Power-up reset
  #2 reset_b = 1;
  #50 reset_b = 0;       // Reset on-the-fly
  #52 reset_b = 1;
  #90 reset_b = 1;
  #92 reset_b = 1;
join

initial fork
  #20 start = 1;
  #30 start = 0;
  #70 start = 1;
  #110 start = 1;
join

```

initial fork

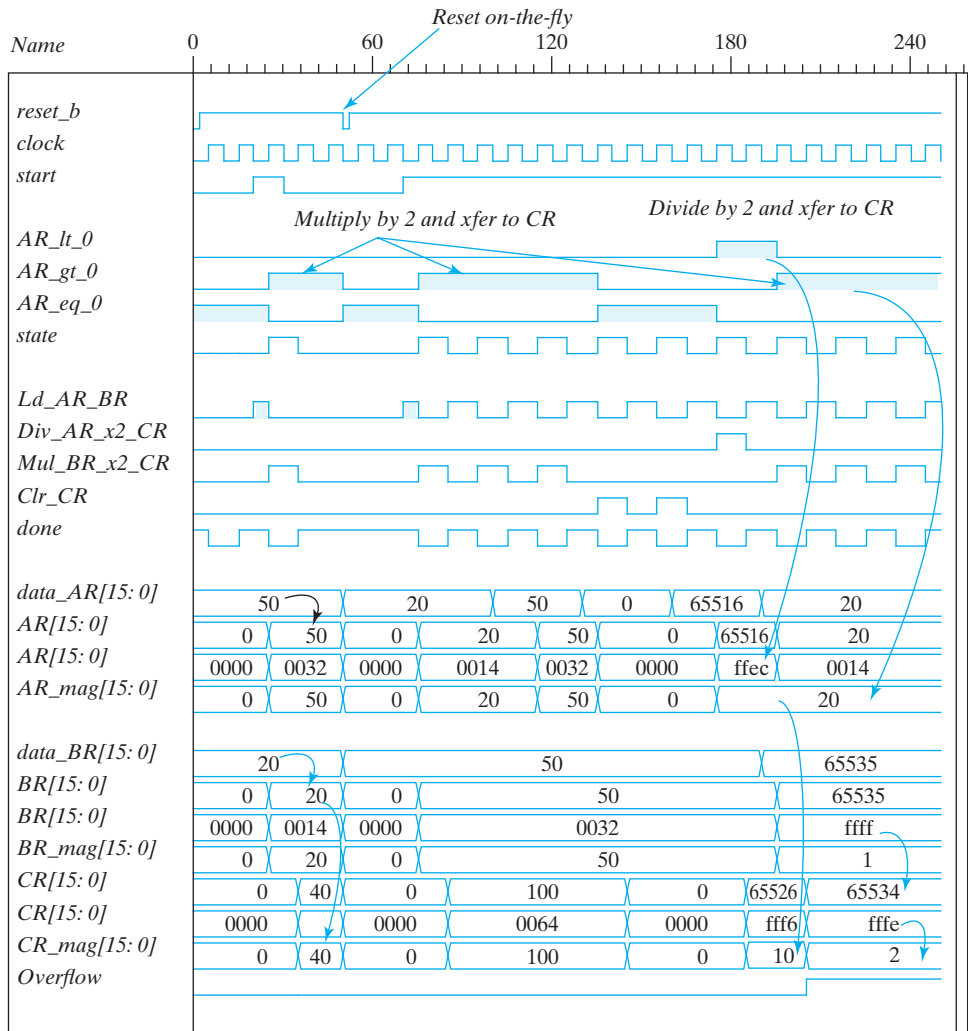
```

data_AR = 16'd50;           // AR > 0
data_BR = 16'd20;           // Result should be 40

#50 data_AR = 16'd20;
#50 data_BR = 16'd50;       // Result should be 100
#100 data_AR = 16'd50;
#100 data_BR = 16'd50;

#130 data_AR = 16'd0;       // AR = 0, result should clear CR
#160 data_AR = -16'd20;     // AR < 0, Verilog stores 16-bit 2s complement
#160 data_BR = 16'd50;     // Result should have magnitude 10
#190 data_AR = 16'd20;     // AR < 0, Verilog stores 16-bit 2s complement
#190 data_BR = 16'hffff;   // Result should have overflow

```

join**endmodule**

VHDL

```

entity Datapath_Prob_8_8 is
  port (Overflow: out Std_Logic; AR_lt_0, AR_gt_0, AR_eq_0: out Std_Logic; data_AR,
        data_BR: in Std_Logic_Vector (15 downto 0); Ld_AR_BR, Div_AR_x2_CR,
        Mul_BR_x2_CR, Clr_CR, clock, reset_b: in Std_Logic);
end Datapath_Prob_8_8;

architecture Behavioral of Datapath_Prob_8_8 is
  AR_lt_0 <= AR(15);
  AR_gt_0 <= (notAR(15) ) and Reduction_OR(AR(14 downto 0));
  AR_eq_0 = (AR = "0000000000000000");
  process (clock, reset_b) begin
    if reset_b'event and reset_b = '0' then AR <= "00000000"; BR <= "00000000";
      CR <= "0000000000000000";
    elsif clock'event and clock = '1' then
      if Ld_AR_BR = '1' then AR <= data_AR; BR <= data_BR;
      elsif Div_AR_x2_CR = '1' then DR <= AR(15) & AR(15 downto 1);
      elsif Mul_BR_x2_C then Overflow & CR <= BR(14 downto 0) & '0';
      elsif Clr_CR = '1' then CR <= "0000000000000000";
      end if;
    end process;
end Behavioral;

entity Controller_Prob_8_8 is
  port (Ld_AR_BR, Div_AR_x2_CR, Mul_BR_x2_CR, Clr_CR, done: out Std_Logic; start,
        AR_lt_0, AR_gt_0, AR_eq_0, clock, reset_b: in Std_Logic);
end Controller_Prob_8_8;

architecture Behavioral of Controller_Prob_8_8 is
  constant S0 = '0', S1 = '1';
  signal state, next_state: Std_Logic_Vector (1 downto 0);
  process (clock, reset_b) begin
    if reset_b'event and reset_b = 0 then state <= S0;
    elsif clock'event and clock = 1 then state <= next_state;
    end process;

  process (state, start, AR_lt_0, AR_gt_0, AR_eq_0) begin
    Ld_AR_BR <= 0;
    Div_AR_x2_CR <= 0;
    Mul_BR_x2_CR <= 0;
    Clr_CR <= 0;

    case state
    when S0 => if start = 1 then Ld_AR_BR = 1; next_state <= S1;
    when S1 => next_state <= S0; if AR_lt_0 then Div_AR_x2_CR = 1;
      elsif AR_gt_0 then Mul_BR_x2_CR <= 1;
      elsif AR_eq_0 then Clr_CR <= 0;
      end if;
    when others => next_state <= S0;
  end process;

```

```

end process
end Behavioral;

entity Prob_8_8 is
    port (done: out Std_Logic; data_AR, data_BR: in Std_Logic_Vector (15 downto 0);
          start, clock, reset_b: in Std_Logic);
end Prob_8_8;

architecture ASMD of Prob_8_8 is
    component Datapath_Prob_8_8
        port (Overflow, AR_lt_0, AR_gt_0, AR_eq_0, data_AR, data_BR, Ld_AR_BR, Div_
              AR_x2_CR, Mul_BR_x2_CR, Clr_CR, clock, reset_b);
    component Controller_Prob_8_8
        port (Ld_AR_BR, Div_AR_x2_CR, Mul_BR_x2_CR, Clr_CR, done, start, AR_lt_0,
              AR_gt_0, AR_eq_0, clock, reset_b);
    begin

        M0: Controller_Prob_8_8
            port map (Ld_AR_BR, Div_AR_x2_CR, Mul_BR_x2_CR, Clr_CR, done, start, AR_
                     lt_0, AR_gt_0, AR_eq_0, clock, reset_b);
        M1: Datapath_Prob_8_8
            port map (Overflow, AR_lt_0, AR_gt_0, AR_eq_0, data_AR, data_BR, Ld_AR_BR,
                     Div_AR_x2_CR, Mul_BR_x2_CR, Clr_CR, clock, reset_b);
    end ASMD;

    function Reduction_OR (data: std_logic_vector) return std_logic is
        constant all_zeros: std_logic_vector(data'range) := (others => '0');
    begin
        if data = all_zeros then
            return '0';
        else
            return '1';
        end if;
    end Reduction_OR;

```

8.9 Design equations:

$$D_{S_idle} = S_2 + S_idle \text{ Start}'$$

$$D_{S_1} = S_idle \text{ Start} + S_1(A2 \ A3)'$$

$$D_{S_2} = A2 \ A3 \ S_1$$

Verilog

```

module Prob_8_9 (output E, F, output [3: 0] A, output A2, A3, input Start, clock,
reset_b);
    Controller_Prob_8_9 M0 (set_E, clr_E, set_F, clr_A_F, incr_A, Start, A2, A3, clock,
reset_b);
    Datapath_Prob_8_9 M1 (E, F, A, A2, A3, set_E, clr_E, set_F, clr_A_F, incr_A, clock,
reset_b);
endmodule

```

```
// Structural version of the controller (one-hot)
// Note that the flip-flop for S_idle must have a set input and reset_b is wire to the set
// Simulation results match Fig. 8-13
```

```
module Controller_Prob_8_9 (
    output set_E, clr_E, set_F, clr_A_F, incr_A,
    input Start, A2, A3, clock, reset_b
);

    wire    D_S_idle, D_S_1, D_S_2;
    wire    q_S_idle, q_S_1, q_S_2;
    wire    w0, w1, w2, w3;
    wire [2:0] state = {q_S_2, q_S_1, q_S_idle};

    // Next-State Logic
    or (D_S_idle, q_S_2, w0);           // input to D-type flip-flop for q_S_idle
    and (w0, q_S_idle, Start_b);
    not (Start_b, Start);

    or (D_S_1, w1, w2, w3);           // input to D-type flip-flop for q_S_1
    and (w1, q_S_idle, Start);
    and (w2, q_S_1, A2_b);
    not (A2_b, A2);
    and (w3, q_S_1, A2, A3_b);
    not (A3_b, A3);

    and (D_S_2, A2, A3, q_S_1); // input to D-type flip-flop for q_S_2

    D_flop_S M0 (q_S_idle, D_S_idle, clock, reset_b);
    D_flop M1 (q_S_1, D_S_1, clock, reset_b);
    D_flop M2 (q_S_2, D_S_2, clock, reset_b);

    // Output Logic
    and (set_E, q_S_1, A2);
    and (clr_E, q_S_1, A2_b);
    buf (set_F, q_S_2);
    and (clr_A_F, q_S_idle, Start);
    buf (incr_A, q_S_1);
endmodule

module D_flop (output reg q, input data, clock, reset_b);
    always @ (posedge clock, negedge reset_b)
        if (!reset_b) q <= 1'b0; else q <= data;
endmodule

module D_flop_S (output reg q, input data, clock, set_b);
    always @ (posedge clock, negedge set_b)
        if (!set_b) q <= 1'b1; else q <= data;
endmodule
```

```

/*
// RTL Version of the controller
// Simulation results match Fig. 8-13

module Controller_Prob_8_9 (
    output reg    set_E, clr_E, set_F, clr_A_F, incr_A,
    input         Start, A2, A3, clock, reset_b
);
    parameter S_idle = 3'b001, S_1 = 3'b010, S_2 = 3'b100; // One-hot
    reg [2: 0] state, next_state;

    always @ (posedge clock, negedge reset_b)
        if (!reset_b) state <= S_idle; else state <= next_state;

    always @ (state, Start, A2, A3) begin
        set_E      = 1'b0;
        clr_E      = 1'b0;
        set_F      = 1'b0;
        clr_A_F    = 1'b0;
        incr_A     = 1'b0;
        case (state)
            S_idle:  if (Start) begin next_state = S_1; clr_A_F = 1; end
                     else next_state = S_idle;

            S_1:     begin
                       incr_A = 1;
                       if (!A2) begin next_state = S_1; clr_E = 1; end
                       else begin
                           set_E = 1;
                           if (A3) next_state = S_2; else next_state = S_1;
                           end
                       end

            S_2:     begin next_state = S_idle; set_F = 1; end
            default: next_state = S_idle;
        endcase
    end
endmodule

*/

module Datapath_Prob_8_9 (
    output reg E, F, output reg [3: 0] A, output A2, A3,
    input set_E, clr_E, set_F, clr_A_F, incr_A, clock, reset_b
);
    assign A2 = A[2];
    assign A3 = A[3];

```

```

always @ (posedge clock, negedge reset_b) begin
  if (!reset_b) begin E <= 0; F <= 0; A <= 0; end
  else begin
    if (set_E) E <= 1;
    if (clr_E) E <= 0;
    if (set_F) F <= 1;
    if (clr_A_F) begin A <= 0; F <= 0; end
    if (incr_A) A <= A + 1;
  end
end
endmodule

```

// Test Plan - Verify: (1) Power-up reset, (2) match ASMD chart in Fig. 8-9 (d),
 // (3) recover from reset on-the-fly

```

module t_Prob_8_9;
  wire E, F;
  wire [3: 0] A;
  wire A2, A3;
  reg Start, clock, reset_b;

  Prob_8_9 M0 (E, F, A, A2, A3, Start, clock, reset_b);

  initial #500 $finish;
  initial begin clock = 0; forever #5 clock = ~clock; end
  initial begin reset_b = 0; #2 reset_b = 1; end
  initial fork
    #20 Start = 1;
    #40 reset_b = 0;
    #62 reset_b = 1;
  join
endmodule

```

VHDL

```

entity Datapath_Prob_8_9 is
  port (E, F: out Std_Logic; A: out Std_Logic_Vector (3 downto 0); A2, A3: out Std_
    Logic; set_E, clr_E, set_F, clr_A_F, incr_A, clock, reset_b: in Std_Logic);
end Datapath_Prob_8_9;

architecture Behavioral of Datapath_Prob_8_8 is
  A2 <= A(2);
  A3 <= A(3);
  process (clock, reset_b) begin

```

```

if reset_b'event and reset_b = '0' then E <= '0', F <= '0', A <= '0';
else
    if set_E = '1' then E <= '1'; end if;
    if clr_E = '1' then E <= '0'; end if;
    if set_F = '1' then F <= '1'; end if;
    if clr_a_F = '1' then A = '0'; F <= '0'; end if;
    if incr_A = '1' then A <= A + "0001"; end if;
end if;
end process;
end Behavioral;

entity Controller_Prob_8_9 is
    port (set_E, clr_E, set_F, clr_A_F, incr_A: out Std_Logic; Start, A2, A3, clock,
        reset_b: in Std_Logic);
end Controller_Prob_8_9;

architecture Behavioral of Controller_Prob_8_9 is
    constant S_idle: Std_Logic := "001"; -- One-Hot
    constant S_1: Std_Logic := "010";
    constant D_2: Std_Logic := "100";
    signal state, next_state: Std_Logic_Vector (2 downto 0);

process (clock, reset_b) begin
    if reset_b'event and reset_b = 0 then state <= S_idle;
    elsif clock'event and clock = 1 then state <= next_state;
end process;

process (state, Start, A2, A3) begin
    set_E <= 0;
    clr_E <= 0;
    set_F <= 0;
    clr_A_F <= 0;
    incr_A <= 0

    case state
        when S_idle => if Start = '1' then clr_A_F = '1'; next_state <= S1; end if;
        when S_1 => incr_A <= '1'; if not A2 then next_state <= S_1; clr_E <= '1';
            else set_E <= '1'; if A3 = '1' then next_state <= S_2 else next_state <= S_1;
            end if;
            end if;
        when S_2 => next_state <= S_idle; set_F <= '1';
        when others => next_state <= S_idle;
    end case;

```



```

        elsif AR_gt_0 = '1' then Mul_BR_x2_CR <= 1;
        elsif AR_eq_0 = '1' then Clr_CR <= 0;
        end if;
        when others => next_state <= S0;
    end process
end Behavioral;

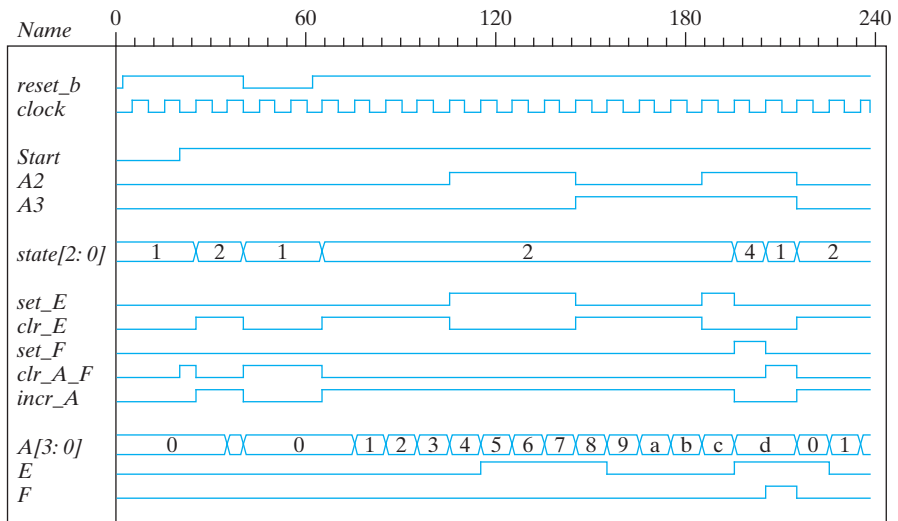
entity Prob_8_9 is
    port (E, F: out Std_Logic; A: out Std_Logic_Vector (3 downto 0); A2, A3: out
          Std_Logic; Start,
          clock, reset_b: in Std_Logic);
end Prob_8_9;

architecture ASMD of Prob_8_9 is
    component Datapath_Prob_8_9
        port (E, F: out Std_Logic; A: out Std_Logic_Vector (3 downto 0); A2, A3: out
              Std_Logic; set_E, clr_E, set_F, clr_A_F, incr_A, clock, reset_b: in Std_Logic);
    component Controller_Prob_8_9
        port (
    );
begin

M0: Controller_Prob_8_9
    port map (set_E, clr_E, set_F, clr_A_F, incr_A, Start, A2, A3, clock, reset_b);

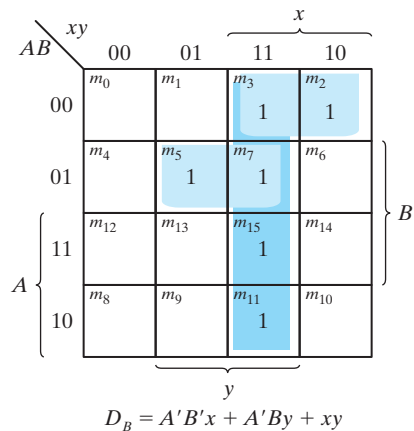
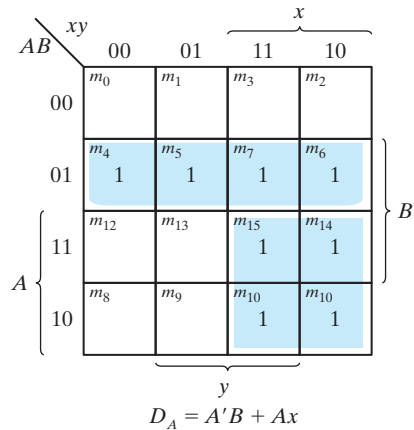
M1: Datapath_Prob_8_9
    port map (E, F, A, A2, A3, set_E, clr_E, set_F, clr_A_F, incr_A, clock, reset_b);
end ASMD;

```



8.11 $D_A = A'B + Ax$
 $D_B = A'B'x + A'By + xy$

State	Inputs	Next state
0 0	0 0	0 0
0 0	0 1	0 0
0 0	1 0	0 1
0 0	1 1	0 1
0 1	0 0	1 0
0 1	0 1	1 1
0 1	1 0	1 0
0 1	1 1	1 1
1 0	0 0	0 0
1 0	0 1	0 0
1 0	1 0	1 0
1 0	1 1	1 1
1 1	0 0	0 0
1 1	0 1	0 0
1 1	1 0	1 0
1 1	1 1	1 1



8.16 RTL notation:

$s0$: (initial state) If $start = 0$ go back to state $s0$, If ($start = 1$) then $BR \leftarrow multiplicand$, $AR \leftarrow multiplier$, $PR \leftarrow 0$, go to $s1$.

$s1$: (check AR for Zero) $Zero = 1$ if $AR = 0$, if ($Zero = 1$) then go back to $s0$ (done) If ($Zero = 0$) then go to $s1$, $PR \leftarrow PR + BR$, $AR \leftarrow AR - 1$.

The internal architecture of the datapath consists of a double-width register to hold the product (PR), a register to hold the multiplier (AR), a register to hold the multiplicand (BR), a double-width parallel adder, and single-width parallel adder. The single-width adder is used to implement the operation of decrementing the multiplier unit. Adding a word consisting entirely of 1's to the multiplier accomplishes the 2's complement subtraction of 1 from the multiplier. Figure 8.16(a) below shows the ASMD chart, block diagram, and controller of othe

circuit. Figure 8.16(b) shows the internal architecture of the datapath. Figure 8.16(c) shows the results of simulating the circuit.

Verilog

```

module Prob_8_16_STR (
output [15: 0] PR, output done,
input [7: 0] data_AR, data_BR, input start, clock, reset_b
);

    Controller_P8_16 M0 (done, Ld_regs, Add_decr, start, zero, clock, reset_b);

    Datapath_P8_16 M1 (PR, zero, data_AR, data_BR, Ld_regs, Add_decr, clock, reset_b);
endmodule

module Controller_P8_16 (output done, output reg Ld_regs, Add_decr, input start,
zero, clock, reset_b);
    parameter s0 = 1'b0, s1 = 1'b1;
    reg state, next_state;
    assign done = (state == s0);

    always @ (posedge clock, negedge reset_b)
    if (!reset_b) state <= s0; else state <= next_state;

    always @ (state, start, zero) begin
        Ld_regs = 0;
        Add_decr = 0;
        case (state)
            s0: if (start) begin Ld_regs = 1; next_state = s1; end
            s1: if (zero) next_state = s0; else begin next_state = s1; Add_decr = 1; end
            default: next_state = s0;
        endcase
    end
endmodule

module Register_32 (output [31: 0] data_out, input [31: 0] data_in, input clock,
reset_b);
    Register_8 M3 (data_out [31: 24] , data_in [31: 24], clock, reset_b);
    Register_8 M2 (data_out [23: 16] , data_in [23: 16], clock, reset_b);
    Register_8 M1 (data_out [15: 8] , data_in [15: 8], clock, reset_b);
    Register_8 M0 (data_out [7: 0] , data_in [7: 0], clock, reset_b);
endmodule

module Register_16 (output [15: 0] data_out, input [15: 0] data_in, input clock,
reset_b);
    Register_8 M1 (data_out [15: 8] , data_in [15: 8], clock, reset_b);
    Register_8 M0 (data_out [7: 0] , data_in [7: 0], clock, reset_b);
endmodule

module Register_8 (output [7: 0] data_out, input [7: 0] data_in, input clock, reset_b);
    D_flop M7 (data_out[7] data_in[7], clock, reset_b);

```

```

D_flop M6 (data_out[6] data_in[6], clock, reset_b);
D_flop M5 (data_out[5] data_in[5], clock, reset_b);
D_flop M4 (data_out[4] data_in[4], clock, reset_b);
D_flop M3 (data_out[3] data_in[3], clock, reset_b);
D_flop M2 (data_out[2] data_in[2], clock, reset_b);
D_flop M1 (data_out[1] data_in[1], clock, reset_b);
D_flop M0 (data_out[0] data_in[0], clock, reset_b);
endmodule

module Adder_32 (output c_out, output [31: 0] sum, input [31: 0] a, b);
    assign {c_out, sum} = a + b;
endmodule

module Adder_16 (output c_out, output [15: 0] sum, input [15: 0] a, b);
    assign {c_out, sum} = a + b;
endmodule

```

VHDL

```

entity Datapath_Prob_8_16 is
    port (PR: out Std_Logic_Vector (15 downto 0) downto 0); zero: out Std_Logic;
    data_AR, data_BR: in Std_Logic_Vector (7 downto 0); Ld_regs, Add_decr, clock,
    reset_b: in Std_Logic;
end Datapath_Prob_8_16;

architecture Behavioral of Datapath_Prob_8_16 is
    zero <= not Reduction_OR(AR);
process (clock, reset_b) begin
    if reset_b'event and reset_b = '0' then AR <= "00000000"; BR <= "0000000000000000";
    elsif clock'event and clock = '1' then
        if Ld_regs = '1' then AR <= data_AR; BR <= data_BR; PR <= X"0000";
        elsif Add_decr = '1' then PR <= PR + BR; AR <= AR -1;

        elsif Mul_BR_x2_C then Overflow & CR <= BR(14 downto 0) & '0';
        elsif Clr_CR = '1' then CR <= "0000000000000000";
        end if;
    end if;
end process;
end Behavioral;

entity Controller_Prob_8_16 is
    port (done: out Std_Logic; Ld_regs, Add_decr: out Std_Logic; start, zero, clock,
    reset_b: in Std_Logic);
end Controller_Prob_8_8;

architecture Behavioral of Controller_Prob_8_16 is
    constant s0 = '0', s1 = '1';
    signal state, next_state: Std_Logic_Vector (1 downto 0);
begin
    done <= state = s0;

```

```

process (clock, reset_b) begin
    if reset_b'event and reset_b = 0 then state <= s0;
    elsif clock'event and clock = 1 then state <= next_state;
end process;

process (state, start, zero) begin
    Ld_regs <= 0;
    Add_decr <= 0;
    case state
        when s0 => if start = 1 then Ld_regs = 1; next_state <= s1; end if;
        when s1 => if zero = '1' then next_state <= s0;
            else next_state <= s1; Add_decr <= '1'; end if;
        when others => next_state <= s0;
    end case;
end process
end Behavioral;

entity Prob_8_16 is
    port (PR: out Std_Logic_Vector (15 downto 0); done: out Std_Logic; data_AR,
        data_BR: in Std_Logic_Vector (7 downto 0); start, clock, reset_b: in Std_Logic);
end Prob_8_8;

architecture ASMD of Prob_8_16 is
component Datapath_Prob_8_16
    port (PR, zero, data_AR, data_BR, Ld_regs, Add_decr, clock, reset_b);

component Controller_Prob_8_16
    port (done, Ld_regs, Add_decr, start, zero, clock, reset_b);
begin

M0: Controller_Prob_8_16
    port map (done, Ld_regs, Add_decr, start, zero, clock, reset_b);

M1: Datapath_Prob_8_16
    port map (PR, zero, data_AR, data_BR, Ld_regs, Add_decr, clock, reset_b);
end ASMD;

function Reduction_OR (data: std_logic_vector) return std_logic is
    constant all_zeros: std_logic_vector(d'range) := (others => '0');
begin
    if data = all_zeros then
        return '0';
    else
        return '1';
    end if;
end Reduction_OR;

module D_flop (output q, input data, clock, reset_b);
always @ (posedge clock, negedge reset_b)
if (!reset_b) q <= 0; else q <= data;
endmodule

```

```

module Datapath_P8_16 (
output reg [15: 0] PR, output zero,
input [7: 0] data_AR, data_BR, input Ld_regs, Add_decr, clock, reset_b
);

reg [7: 0] AR, BR;
assign zero = ~( | AR);

always @ (posedge clock, negedge reset_b)
if (!reset_b) begin AR <= 8'b0; BR <= 8'b0; PR <= 16'b0; end
else begin
if (Ld_regs) begin AR <= data_AR; BR <= data_BR; PR <= 0; end
else if (Add_decr) begin PR <= PR + BR; AR <= AR -1; end
end
endmodule

// Test plan – Verify;
// Power-up reset
// Data is loaded correctly
// Control signals assert correctly
// Status signals assert correctly
// Start is ignored while multiplying
// Multiplication is correct
// Recovery from reset on-the-fly

module t_Prob_P8_16;
wire done;
wire [15: 0] PR;
reg [7: 0] data_AR, data_BR;
reg start, clock, reset_b;

Prob_8_16_STR M0 (PR, done, data_AR, data_BR, start, clock, reset_b);

initial #500 $finish;
initial begin clock = 0; forever #5 clock = ~clock; end
initial fork
reset_b = 0;
#12 reset_b = 1;
#40 reset_b = 0;
#42 reset_b = 1;
#90 reset_b = 1;
#92 reset_b = 1;
join

initial fork
#20 start = 1;
#30 start = 0;
#40 start = 1;
#50 start = 0;
#120 start = 1;
#120 start = 0;
join

```

initial fork

data_AR = 8'd5; // AR > 0

data_BR = 8'd20;

#80 data_AR = 8'd3;

#80 data_BR = 8'd9;

#100 data_AR = 8'd4;

#100 data_BR = 8'd9;

join

endmodule

8.17 $(2^n - 1)(2^n - 1) < (2^{2n} - 1)$ for $n \geq 1$

8.18 (a) The maximum number of bits in the product will be $32 + 32 = 64$.

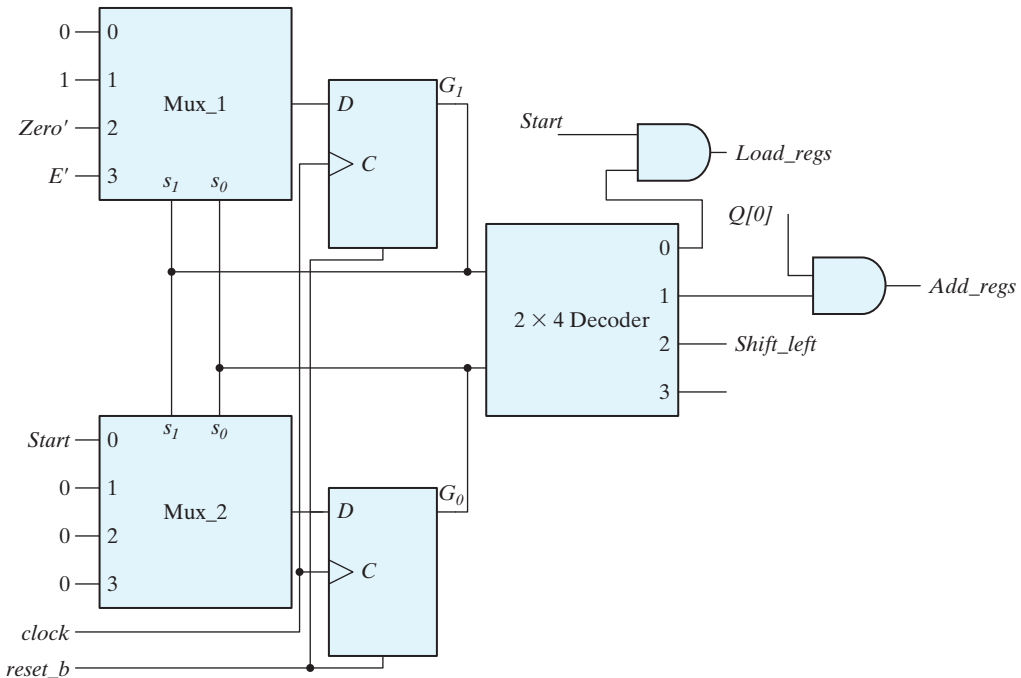
(b) P counter must have 6 bits to load 32 (binary 100000) initially.

(c) The required circuit should output 1 if value in P is 000000. So, a 6-input NOR gate should be included.

8.20 $2(n + 1)t$

8.21

State codes:	G_1	G_0
S_{idle}	0	0
S_{add}	0	1
S_{shift}	1	0
unused	0	0



8.30 Verilog

(a) $RA = 1100, RB = 1000$ (b) $RA = 1100, RB = 1010$

VHDL

(a) $RA = 1100, RB = 1000$ (b) $RA = 1100, RB = 1010$

8.31 $A = 1001, B = 0110, C = 0001$.

$A + B = 1111$

$A - B = 0011$

$A * C = 1001$

$\sim B = 1001$

$!A = !(1) = 0$

$A >> 1 = 0100$

$A >>> 1 = 1100$

$B \& C = 0000$

$B \&\& C = 1$

$B \parallel C = 1$

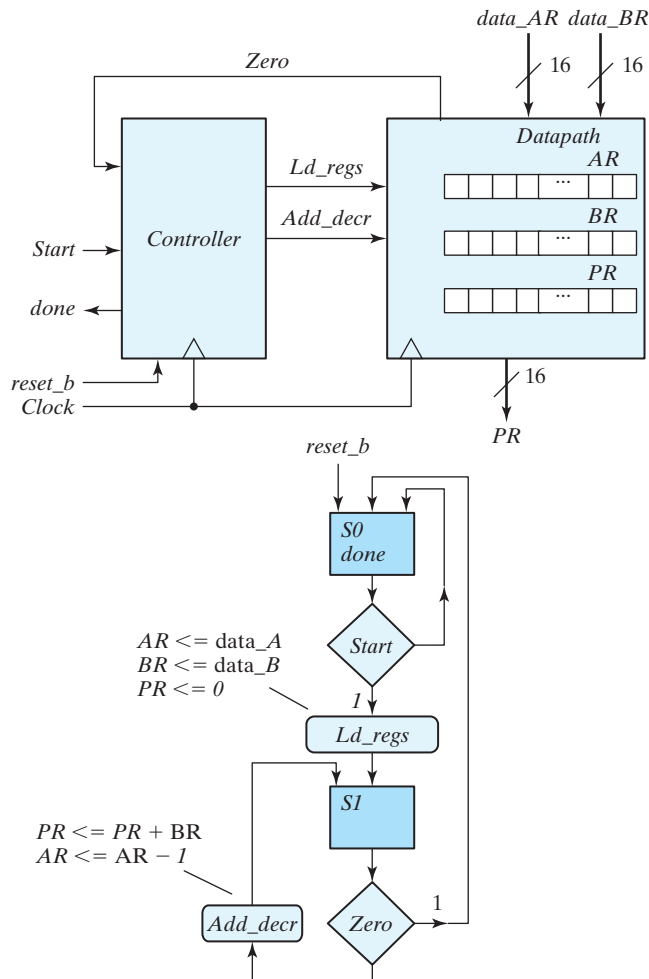
$B | C = 0111$

$A \wedge B = 1111$

$\sim (A | B) = \sim (1111) = 0000$

$\sim (A \& C) = \sim (0000) = 1111$

$A < B = 0$

8.39


Verilog

```

module Prob_8_39 (
    output [15: 0] PR, output done,
    input [7: 0] data_AR, data_BR, input start, clock, reset_b
);

    Controller_P8_39 M0 (done, Ld_regs, Add_decr, start, zero, clock, reset_b);

    Datapath_P8_39 M1 (PR, zero, data_AR, data_BR, Ld_regs, Add_decr, clock,
        reset_b);
endmodule

```

```

module Controller_P8_16 (output done, output reg Ld_regs, Add_decr, input start,
zero, clock, reset_b);
    parameter s0 = 1'b0, s1 = 1'b1;
    reg state, next_state;
    assign done = (state == s0);
    always @ (posedge clock, negedge reset_b)
        if (!reset_b) state <= s0; else state <= next_state;

    always @ (state, start, zero) begin
        Ld_regs = 0;
        Add_decr = 0;
        case (state)
            s0:          if (start) begin Ld_regs = 1; next_state = s1; end
            s1:          if (zero) next_state = s0;
                       else begin next_state = s1; Add_decr = 1; end
            default:    next_state = s0;
        endcase
    end
endmodule

```

```

module Datapath_P8_16 (
    output reg [15: 0] PR, output zero,
    input [7: 0] data_AR, data_BR, input Ld_regs, Add_decr, clock, reset_b
);
    reg [7: 0] AR, BR;
    assign zero = ~( | AR);

    always @ (posedge clock, negedge reset_b)
        if (!reset_b) begin AR <= 8'b0; BR <= 8'b0; PR <= 16'b0; end
        else begin
            if (Ld_regs) begin AR <= data_AR; BR <= data_BR; PR <= 0; end
            else if (Add_decr) begin PR <= PR + BR; AR <= AR -1; end
        end
endmodule

```

```

// Test plan – Verify;
// Power-up reset
// Data is loaded correctly
// Control signals assert correctly
// Status signals assert correctly
// Start is ignored while multiplying
// Multiplication is correct
// Recovery from reset on-the-fly

module t_Prob_P8_16;
    wire done;
    wire [15: 0] PR;
    reg [7: 0] data_AR, data_BR;
    reg start, clock, reset_b;

    Prob_8_16 M0 (PR, done, data_AR, data_BR, start, clock, reset_b);

    initial #500 $finish;
    initial begin clock = 0; forever #5 clock = ~clock; end
    initial fork
        reset_b = 0;
        #12 reset_b = 1;
        #40 reset_b = 0;
        #42 reset_b = 1;
        #90 reset_b = 1;
        #92 reset_b = 1;
    join

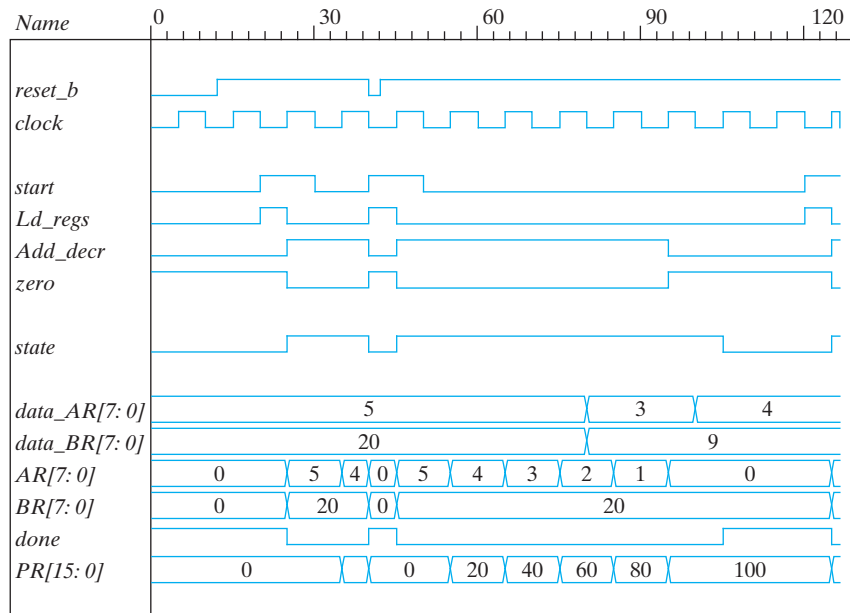
    initial fork
        #20 start = 1;
        #30 start = 0;
        #40 start = 1;
        #50 start = 0;
        #120 start = 1;
        #120 start = 0;
    join

    initial fork
        data_AR = 8'd5; // AR > 0
        data_BR = 8'd20;

        #80 data_AR = 8'd3;
        #80 data_BR = 8'd9;

        #100 data_AR = 8'd4;
        #100 data_BR = 8'd9;
    join
endmodule

```

VHDL

entity Prob_8_39 **is**

port (PR: **out** Std_Logic_Vector (15 **downto** 0); done: **out** Std_Logic;
 data_AR, data_BR: **in** Std_Logic_Vector (7 **downto** 0);
 start, clock, reset_b: **in** Std_Logic);

end Prob_8_39;

architecture ASMD of Prob_8_39 **is**

component Controller_P8_39 (done, Ld_regs, Add_decr: **out** Std_Logic;
 Start, zero, clock, reset_b: **in** Std_Logic);

component Datapath_P8_39 (PR: **out** Std_Logic_Vector (15 **downto** 0); zero:
 out Std_Logic; data_AR, data_BR: **in** Std_Logic_Vector (7 **downto** 0); Ld_regs,
 Add_decr, clock, reset_b: **in** Std_Logic);

begin

M0 Controller_P8_39 **port map** (done, Ld_regs, Add_decr, start, zero, clock, reset_b);

M1 Datapath_P8_39 **port map** (PR, zero, data_AR, data_BR, Ld_regs, Add_decr,
 clock, reset_b);

end Structural;

entity Controller_P8_39 **is**

port (done: **out** Std_Logic; Ld_regs, Add_decr: **out** Std_Logic; start, zero, clock,
 reset_b: **in** Std_Logic);

end Controller_P8_16;

```

architecture Behavioral of Controller_P8_39 is
    constant      s0 = 1'b0, s1 = 1'b1;
    signal        state, next_state;
begin
    done <= (state = s0);

    process (clock, reset_b)
        if reset_b'event and reset_b = 0 then state <= s0;
        elsif clock'event and clock = 1 then state <= next_state; end if;
    end process;

    process (state, start, zero)
        Ld_regs = 0;
        Add_decr = 0;
        case state is
            when s0 =>:      if (start = 1) then Ld_regs = 1; next_state = s1; end if;
            when s1 =>:      if (zero = 1) then next_state = s0; else next_state = s1;
            Add_decr = 1;    end if;
            others:          next_state = s0;
        endcase
    end process;
end Behavioral;

entity Datapath_P8_39 (
    port (PR: out Std_Logic_vector (15 downto 0); zero: out Std_Logic; data_AR,
        data_BR: in Std_Logic_Vector (7 downto 0); input Ld_regs, Add_decr, clock,
        reset_b: in Std_Logic);
end Datapath_P8_16;

architecture Behavioral of Datapath_P8_16 is
begin
    zero <= not( AR(7) or AR(6) or AR(5) or AR(4) or AR(3) or AR(2) or AR(1) or
    AR(0));

    process (clock, reset_b)
    begin
    if reset_b'event and reset_b = 0 then AR <= 8'b0; BR <= 8'b0; PR <= 16'b0;
        elsif clock'event and clock = 1 then
            if Ld_regs = 1 then AR <= data_AR; BR <= data_BR; PR <= 0; end if;
            elsif Add_decr = 1 then PR <= PR + BR; AR <= AR -1; end if;
        end process;
    end Behavioral;

    // Test plan – Verify;
    // Power-up reset
    // Data is loaded correctly
    // Control signals assert correctly
    // Status signals assert correctly
    // Start is ignored while multiplying

```

```
// Multiplication is correct
// Recovery from reset on-the-fly
```

```
entity t_Prob_P8_39 is
end t_Prob_P8_39;
```

```
architecture Test_Bench of t_Prob_P8_39 is
```

```
    signal    t_done: Std_Logic;
    signal    t_PR: Std_Logic_Vector (15 downto 0);
    signal    t_data_AR, t_data_BR: Std_Logic_Vector (7 downto 0);
    signal    t_start, t_clock, t_reset_b: Std_Logic;
    component Prob_8_39 port (PR: out Std_Logic_Vector (15 downto 0); done:
    out Std_Logic; data_AR, data_BR: in Std_Logic_Vector (7 downto 0); start, clock,
    reset_b: in Std_Logic);
```

```
begin
```

```
    M_UUT Prob_8_39 port map (PR => t_PR, done => t_done; data_AR => t_data_
    AR, data_BR => t_data_BR, start => t_start, clock => t_clock, reset_b => t_reset_b);
```

```
    process clock = '0';
    wait for 5 ns clock <= '1';
    wait for 5 ns;
    end process;
```

```
    reset_b = '0';
    reset_b <= '1' after 12 ns;
    reset_b <= '0' after 40 ns;
    reset_b <= '1' after 42 ns;
    reset_b <= '1' after 90 ns;
    reset_b <= '1' after 92 ns;
```

```
    start <= 1 after 20 ns;
    start <= 0 after 30 ns;
    start <= 1 after 40 ns;
    start <= 0 after 50 ns;
    start <= 1 after 120 ns;
    start <= 0 after 120 ns;
```

```
    data_AR <= 8'd5; // AR > 0
    data_BR <= 8'd20;
```

```
    #80 data_AR <= 8'd3 after 80 ns;
    #80 data_BR <= 8'd9 after 80 ns;
```

```
    data_AR <= 8'd4 after 100 ns;
    data_BR <= 8'd9 after 100 ns;
```

```
end Test_Bench;
```